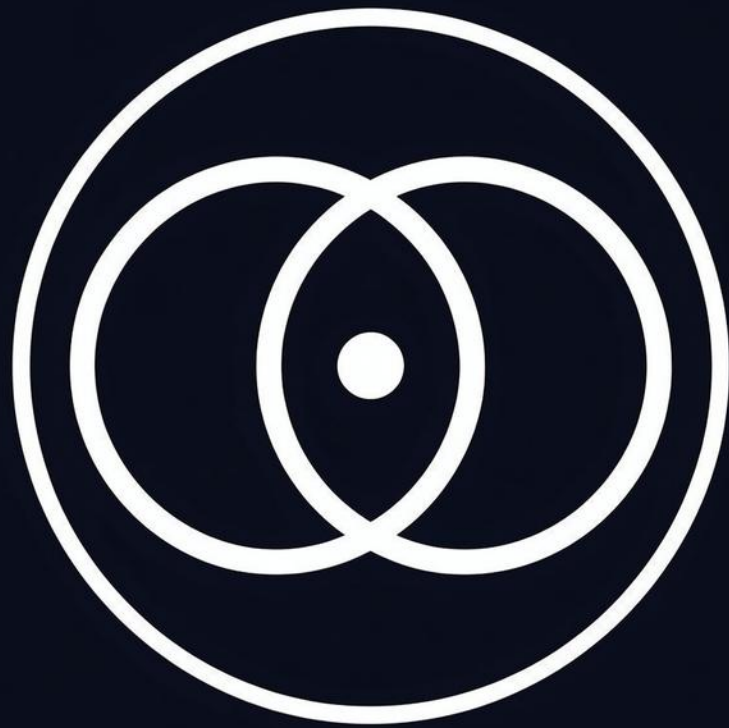




P2

**BFT-lite for Multi-Agent Stop
Decisions: An $n \geq 3f+1$
Committee, Whole-Round
Voiding of Equivocation, and
Safety Verification of View
Change**

多智能体停止决策的 BFT-lite 安全性验证

BFT-lite for Multi-Agent Stop Decisions: An $n \geq 3f+1$ Committee, Whole-Round Voiding of Equivocation, and Safety Verification of View Change

Evidence-grade statement: all assertions in this paper come from the UDOS Reasoning Engine tag **v7.5.0** (commit 1a71270), `udos7/topology/consensus.py`, `tests7/test_v747_consensus.py` (10 items), and `reports7/matrix_scale_demo.json`; a Linux CPU deterministic prototype, uniformly `cpu-proto`; the 10 property tests are verified. Signatures are SHA-256 **hash placeholders**, not cryptographically secure signatures, and do not resist real forgery.

Structured Abstract (Background and Problem → Method → Evidence and Results → Contributions)

[Background and Problem] The "when to stop / rework" decision in a million-agent matrix cannot be made by a single QA point: one wrong vote from a single point either causes an erroneous stop (safety violation, accepting a defective deliverable downstream) or an indefinite wait (liveness violation). Classical BFT assumes errors are independent and random, whereas agent wrong votes have semantic correlation, and directly carrying over full consensus is too costly (see § 1, § 3.1).

[Method] This paper proposes no new consensus algorithm; instead it reduces classical BFT safety/liveness to an "**executable property-test + fault-injection**" suite: committee size $n \geq 3f+1$, quorum $2f+1$, members equivocating (swing voting) in the same round are voided for the whole round, more than f silent members return `NO_QUORUM` and trigger a view change; the constructor layer rejects illegal n (see § 3, § 4).

[Evidence and Results] At the main operating point $n=7, f=2$ ($q=5$), 10 deterministic property tests (verified) cover illegal- n rejection, honest-supermajority stop, Byzantine inability to force a stop, equivocation detection and whole-round voiding, unreachability of conflicting quorums, silence-triggered view change, and rejection of unknown voters; in end-to-end integration (`cpu-proto`, single seed), a 7-member QA committee with 2 Byzantine reverse voters correctly accepted all 24 orders, and a stressed fleet triggered 5 reworks and isolated 3 faulty agents (see § 6).

[Contributions] (i) Reconstruct BFT safety/liveness from "prose proofs" into falsifiable property tests that are CI-able and regression-able in the repository; (ii) give the minimal degraded design of the binary semantic judgment of an "acceptance committee" (no need for full state-machine replication); (iii) honestly state three boundaries — SHA-256 hash-placeholder signatures are not cryptographically secure, network partitions and adaptive adversaries are not modeled, and semantic common-cause failures can break through the majority vote. All assertions are point estimates of a CPU deterministic prototype (see § 7, § 8).

Abstract

The "when to stop / rework" decision in a million-agent matrix cannot rest on a single QA point: a single point will, due to one wrong judgment, either stop erroneously (safety violation) or wait indefinitely

(liveness violation). This paper implements and validates a lightweight Byzantine-fault-tolerant committee, BFT-lite: committee size satisfies $n \geq 3f+1$, quorum $2f+1$, members equivocating on the same decision in the same round are voided for the whole round, and more than f silent members return `NO_QUORUM` and trigger a view change rather than hanging. This paper proposes no new consensus algorithm; instead it reduces classical BFT safety/liveness to an **"executable property-test + fault-injection"** suite suited to semantic agent decisions. Under $n=7, f=2$, 10 deterministic property tests cover: constructor rejection of illegal n , honest-supermajority stop, Byzantine inability to force a wrong stop, equivocation detection and whole-round voiding, unreachability of conflicting quorums, silence-triggered view change, and rejection of unknown voters. In end-to-end integration, a 7-member QA committee with 2 Byzantine reverse voters correctly accepted all 24 orders; a stressed fleet triggered 5 reworks and isolated 3 faulty agents. This paper argues that for agent stop decisions, BFT safety/liveness should be presented as falsifiable property tests, not only as prose proofs. It must be noted that the signatures here are hash placeholders, network partitions and semantic common-cause failures are not modeled, and all assertions are point estimates of a CPU deterministic prototype; we do not claim resistance to real forgery or an open-consensus setting.

Keywords: Byzantine fault tolerance; stop decision; quorum; equivocation; view change; multi-agent systems; property-based testing

1 Introduction

At the end of a multi-agent matrix there is a seemingly trivial but critical decision: **is an order actually "done" (stop), or "not done well, rework" (continue)?** In a small system this decision can be given to a QA agent. But in a million-agent matrix, a single-point QA is a single point of failure: one wrong judgment either misjudges a not-done result as passing (erroneous stop, safety violation) or misjudges a done result as not passing (indefinite wait, liveness violation).

Classical distributed systems long ago answered the prototype of this question: Byzantine-fault-tolerant (BFT) consensus tells us that when at most f members act maliciously, as long as the committee is large enough ($n \geq 3f+1$), a balance can be struck between "consistent decision" and "liveness." But directly moving classical BFT onto agent stop decisions has a subtle difference: **classical BFT assumes replicated state machines and a message round-trip model, with errors random and independent; whereas agent decisions are semantic judgments** — when a QA agent says "rework," the wrong vote has semantic correlation (it may have been misled by the same erroneous context as another QA).

This paper proposes no new consensus algorithm. Our claim is more restrained: **turn BFT safety/liveness from "prose proofs in a paper" into "executable, falsifiable property tests in the repository."** `udos7/topology/consensus.py` implements a minimal BFT-lite: $n \geq 3f+1$, $2f+1$ quorum, whole-round voiding of equivocation, more than f silent members triggering a view change; `tests/test_v747_consensus.py` fixes these properties as regression assertions with 10 deterministic tests. In end-to-end matrix integration, a 7-member QA committee held 24/24 correct acceptances under 2 Byzantine reverse voters.

Design trade-offs of a minimal committee. Why $n=7, f=2$ rather than larger? Because it is one of the smallest legal committees to "tolerate 2 Byzantine members" ($3f+1=7$). It is small enough to run all voting combinations clearly on a CPU, yet large enough to demonstrate the three faults of reverse voting, equivocation, and silence. A larger n (e.g., 10, 13) is only a linear magnification of the same mechanism with unchanged safety analysis; this paper chooses the smallest demonstrable configuration so that each test can be read by the human eye as to "why this assertion holds." The larger

the committee, the stronger BFT's fault tolerance (more f tolerated), but voting cost increases linearly — this is an engineering trade-off between safety and cost, and this paper does not claim any single n is optimal.

The boundaries of this paper must also be stated up front: signatures are SHA-256 **hash placeholders**, not cryptographically secure signatures; network partitions, message reordering, and adaptive adversaries are not modeled; semantic common-cause failures ($>f$ members systematically misled by the same erroneous context) will make the BFT assumption fail — which is exactly what this paper lists as future work.

Why the "stop decision" deserves its own BFT. In a multi-agent pipeline, "stop" is a consequential action: once stopped, the order is accepted, settled, and archived; once erroneously stopped, a defective deliverable enters downstream. By contrast, "continue/rework" is a relatively conservative action — it just makes the order run one more round, at the cost of time. This asymmetry means we must especially guard against "erroneous stop" (safety), while tolerating some cost for "excessive waiting" (liveness). The BFT-lite design is precisely the embodiment of this asymmetry: be conservative on safety (rather rework than wrongly stop), and set a view change as the liveness fallback (do not let the system permanently freeze). This is in line with classical BFT's trade-off of "consistency over availability," but in the agent scenario "consistency" is concretized as "not erroneously accepting."

Intended readers of this paper. This paper addresses two kinds of readers: engineers doing agent orchestration/pipelines, who care about "how the end stop decision is not ruined by a single point"; and researchers of distributed reliable systems, who care about "how classical BFT is degraded and verified in semantic-judgment agent scenarios." For the former we give a directly reusable minimal committee configuration and property tests; for the latter we point out semantic common-cause failure, the deviation from classical assumptions, as an open problem.

2 Related Work

Classical BFT consensus. The foundation and engineering of this field come from Castro and Liskov's PBFT at OSDI 1999: its pre-prepare/prepare/commit three-phase state-machine replication tolerates $f < n/3$ Byzantine nodes under asynchronous networks (i.e., $n \geq 3f+1$), and it reports its BFS implementation to be only about 3% slower than unreplicated NFS. This paper's $n \geq 3f+1$, $2f+1$ quorum, and view change directly reuse the PBFT framework. HotStuff (Yin, Malkhi, Reiter et al., PODC 2019, arXiv:1803.05069) further optimizes BFT to linear communication complexity, responsiveness, and chained view changes, and is the basis of Libra/LibraBFT. This paper does not improve these two algorithms: we neither pursue HotStuff's linear communication complexity nor do full synchronization of state-machine replication, but take only the minimal skeleton of "quorum + Byzantine tolerance + view change" for the agent's binary stop judgment.

Agent voting and multi-agent decision. Multiple LLM agents making decisions through dialogue/voting already have mature frameworks; Wu et al.'s 2023 AutoGen (arXiv:2308.08155) supports mixed LLM/human/tool dialogue; but its close-reading notes also point out that "dialogue without termination conditions easily loops forever." This paper precisely targets this "termination condition": AutoGen-like frameworks let multiple agents dialogue but rarely ask "when does the dialogue stop, and does the stop decision itself need fault tolerance." Taking Byzantine fault tolerance of the agent stop decision as a first-class property is this paper's entry point. Specific LLM voting-decision literature is to be supplemented [CITATION NEEDED: LLM agent voting decision, to be verified].

Property tests and fault injection. Using property tests to enumerate inputs and exhaust small-scale state spaces to verify system invariants is a standard method of reliable engineering [CITATION NEEDED: property-based testing hypothesis, to be verified]. This paper uses it for BFT safety/liveness assertions: translating "not erroneously stopping under any fault strategy" into 10 CI-able deterministic tests.

Relation to recent work on "agent reliability." Recent reliability research on multi-agent systems mostly focuses on "how to keep agents from crashing and make them observable," and less often takes Byzantine fault tolerance of the end "stop/rework" decision as a first-class property. This paper's entry is precisely this gap: at the end of the agent pipeline, "when to stop" is itself a distributed decision requiring fault tolerance. This paper introduces no new fault-tolerance mechanism, but lands the mature mechanisms of classical BFT on this new decision point and verifies them with executable tests. This approach of "moving mature mechanisms to a new decision point + backing it with property tests" is more suitable for the current stage than inventing new algorithms.

The positioning of this paper: not new consensus, but **"reducing BFT safety/liveness to an executable property-test suite in the agent stop-decision scenario."**

Key differences from classical BFT. Classical BFT's replicated state machine requires all replicas to perform exactly the same state transitions and agree on the result; whereas the agent stop decision does not need "all replicas computing the same state," it only needs "the committee reaching an agreeing majority on one binary semantic judgment (stop/not stop)." This difference brings three simplifications: first, we do not need full state synchronization in view changes, only "re-vote"; second, our quorum is binary (STOP/CONTINUE), not an arbitrary value; third, our fault model is semantic — a Byzantine member does not randomly tamper with state but "deliberately gives a wrong acceptance judgment." The third point is both a simplification and a threat: the simplification is that we do not need to model inconsistency among state machines, and the threat is that semantic wrong votes may be correlated (see common-cause failures in Section 7).

Why use property tests rather than pure proofs. Formal proofs can give asymptotic guarantees, but one off-by-one in the engineering implementation (e.g., writing the quorum as $2f+1$ rather than $2f+1$, or not voiding an equivocator for the whole round) makes the proof come to nothing. Property tests translate the "proof" into "enumerating voting combinations and verifying invariants are not violated": it does not prove the algorithm correct, but proves that "in our concrete implementation, the given family of fault strategies triggers no safety counterexample." For an engineering team, the latter is runnable, CI-able, and regression-able. This paper therefore places the 10 tests at the same level of importance as proofs.

2.1 Comparison with template paradigms (NSA naming critique / AlexNet fair self-disclosure)

- **Comparison with NSA's "illusion/myth" naming critique:** NSA uses "The Illusion of Efficient Inference" and "The Myth of Trainable Sparsity" to nail down prior sparse methods, then force out its own design space. This paper dually names "a single QA point deciding to stop" as the **single-point-of-failure illusion** — it "looks able to stop correctly" in small systems, but in a million matrix one wrong judgment is a safety/liveness violation; this paper therefore upgrades the stop decision to a committee and places the most dangerous counterexample of "erroneous stop" (5 honest CONTINUE vs 2 Byzantine STOP, test #3) directly into the property tests.
- **Comparison with AlexNet's "fairness self-disclosure":** AlexNet footnote 2 actively admits that the one-GPU comparison "favors larger networks." This paper dually actively discloses two things unfavorable to itself: (i) signatures are SHA-256-truncated **hash placeholders** that do not resist real

forgery; (ii) the **full-factorial comparison** with single-point QA and naive majority voting has not yet been done, so "how much better BFT-lite is than a single point" currently has only a mechanism argument and no comparison data (§ 7).

- **The " $n \geq 3f+1$ counterexample" is this paper's ablation:** the 10 tests of § 6 are an item-by-item ablation of "whether the system is still correct after removing/violating some invariant" — test #1 violates $n \geq 3f+1$ ($n=6, f=2$) and the constructor must reject; test #5 simulates the break of "an equivocator not voided for the whole round," with the quorum broken to `NO_QUORUM`. This is isomorphic to AlexNet's "removing any convolutional layer gives -2% ": **whenever one claims "X guarantees safety," one must give the counterexample of "how safety breaks after violating/removing X."**

3 Problem Definition and Assumptions

3.1 System and invariants

Committee $V=\{v_1, \dots, v_n\}$, with at most f Byzantine members. Each round `round_id`, members vote on `STOP` (accept) or `CONTINUE` (rework). The decision function `decide(round_id)` returns:

- `STOP` / `CONTINUE`: one side's honest votes reach the quorum $q=2f+1$;
- `NO_QUORUM`: neither side reaches the quorum, with a `reason` (`view_change` or `waiting`);
- `safety_violation`: if both sides **simultaneously** gather a quorum, return `None` + `safety_violation=True`.

Two target invariants:

- **Safety**: at most one decision value. When $n \geq 3f+1$, honest votes are only $n-f \geq 2f+1=q$, making it impossible for `STOP` and `CONTINUE` to **each** gather q honest votes.
- **Liveness**: no silent hang. When the number of silent (not voted) members is $>f$, return `NO_QUORUM(view_change)` rather than waiting indefinitely; $>f$ silence means it may not even gather a quorum, and the view should change.

Why a "stop decision" rather than arbitrary consensus. We limit the decision space to binary (`STOP`/`CONTINUE`), not arbitrary state values. This limitation has two benefits: first, the safety analysis of binary quorums is clearer — as long as the two sides cannot simultaneously reach q ; second, it fits the real end of an agent pipeline: an order is either accepted or reworked, with no need to choose an arbitrary value among multiple candidates. This also means this paper's BFT-lite is not a general replicated state machine but an "acceptance committee." Misreading it as "applicable to arbitrary distributed consensus" goes beyond bounds, and we explicitly do not claim that.

3.2 Assumptions (falsifiable)

- **H1 (safety)**: when honest votes $\geq 2f+1$, the probability of a wrong decision is 0; conflicting quorums are possible only when equivocation is not detected, and after detection the whole round is voided. **Falsification**: any fault strategy with $f \leq (n-1)/3$ producing a wrong `STOP` overturns H1.
- **H2 (liveness)**: when silence $>f$, return `NO_QUORUM/view_change` rather than hanging; without silence, `STOP` or `CONTINUE` within finite rounds.
- **H3 (illegal-configuration rejection)**: when $n < 3f+1$, the constructor raises `ValueError`, eliminating illegal configurations from the API layer.

Note on the falsifiability of the assumptions. All three assumptions are "zero-tolerance": H1 requires the probability of an erroneous stop to be 0, falsified by a single counterexample; H2 requires no hang,

falsified by a single configuration that is forever NO_QUORUM without view change; H3 requires constructor rejection, falsified by being able to construct a committee with $n < 3f + 1$. Such zero-tolerance assumptions fit deterministic systems — they do not allow "safe on average" but require "safe under any concrete input." This is also why we use property tests rather than statistical tests: what we seek is "counterexample existence," not "average effect."

4 Method and System Design

4.1 Voting and signature placeholders (`consensus.py`)

`sign(voter, decision, round_id)` is `sha256(f"{voter}|{decision}|{round_id}")[:16]` — this is a **hash placeholder**, used only for deterministic recording of same-round votes, not a cryptographic signature. `cast(voter, decision, round_id)` rejects unknown voters (`unknown voter`).

Why honestly label it a "hash placeholder." A common engineering mistake is to mistake "able to compute a deterministic string" for "forgery-resistant signature." This paper's `sign` only hash-truncates the voting triple, and anyone knowing the rule can generate a legal hash for any voter — it provides no unforgeability. We keep it because it keeps voting records deterministic and reproducible in controlled simulation (`test_signature_is_deterministic` verifies same input gives same signature, different decisions give different signatures), but it does **not** correspond to cryptographic-signature infrastructure in a real network. Writing this clearly prevents readers from misreading "deterministic voting records in simulation" as "unforgeable voting in a real network."

4.2 Equivocation detection and whole-round voiding

`equivocators(round_id)` finds members voting on **multiple** decisions in the same round (swingers/double voters). `decide` **voids these members for the whole round**: their votes are not counted. Other members get one vote counted by their **first** vote this round (`bs[0].decision`). This design corresponds to classical BFT's treatment of equivocation: a swinger cannot "count one vote on each side" to gather a quorum.

Why "void for the whole round" rather than "take only the last vote." There is a subtle choice here: when a member votes both STOP and CONTINUE in the same round, do we "take the last vote" or "void for the whole round"? This paper chooses whole-round voiding (counting none of that member's votes this round). The reason: if only the last vote is taken, a Byzantine member could first vote the side favorable to itself, then change votes according to the situation, manipulating the quorum midway; whole-round voiding eliminates this swing manipulation — once you swing, you have no voice this round. Tests #4 and #5 are the two sides of this design: in #4, after the swinger is voided, the remaining honest votes still reach quorum (STOP=5) and decide normally; in #5, after the swinger is voided, honest votes are fewer than 5 and turn to NO_QUORUM. Both verify "swingers are not trusted."

4.3 Quorum and view change

```
decide(round_id):
    equiv = equivocators(round_id)          # detect swingers
    tally = {STOP:0, CONTINUE:0}
    for v in voters:
        if v in equiv: continue           # void for the whole round
        bs = v's votes this round
        if bs: tally[bs[0].decision] += 1
    silent = members not voting
    if tally[STOP] ≥ q and tally[CONTINUE] ≥ q:
        return None, safety_violation=True, "conflicting_quorums"
    for d in (STOP, CONTINUE):
        if tally[d] ≥ q: return d, "quorum"
    return NO_QUORUM, ("view_change" if len(silent) > f else "waiting")
```

Key: `safety_violation` triggers only when **both sides reach quorum** — and under $n \geq 3f+1$, honest votes cannot make both sides reach q , so this branch **should not be triggered by honest voting** under legal configurations; its meaning is to clearly error once triggered (indicating an anomaly detected, such as undetected equivocation).

4.3.1 Safety proof sketch (why $n \geq 3f+1$)

Let the committee have n members, at most f Byzantine. Honest members are at least $n-f$. Quorum $q=2f+1$:

$$q = 2f+1, \quad n-f \geq q = 2f+1 \implies n \geq 3f+1$$

To have STOP and CONTINUE **each** gather q honest votes requires at least $2q=4f+2$ honest votes; but the total honest votes is only $n-f$. Setting $n-f \geq 4f+2$ would satisfy both sides, i.e., $n \geq 5f+2$ — stricter than $3f+1$. The classical result $n \geq 3f+1$ suffices because it allows Byzantine members to **vote only one side** (rather than q on each side): at $n \geq 3f+1$, $n-f \geq 2f+1=q$, honest votes suffice for one side to reach quorum, while the Byzantine f votes push the other side to at most $f < q$, unable to make both sides reach q . The harm of equivocation is that a swinger may "count one vote on each side," artificially gathering a second quorum; this paper eliminates this loophole through whole-round voiding. This is why the constructor enforces $n \geq 3f+1$: below it, honest votes are insufficient to hold the unique decision value under Byzantine reverse voting.

4.3.2 The boundary of liveness

Liveness is more subtle than safety. If more than f members are silent (not voting), then the honest countable members may be fewer than $q=2f+1$, at which point neither side can reach quorum. This paper's handling: when the silent count is $>f$, return `NO_QUORUM(reason=view_change)`, indicating "the current view cannot gather a quorum and should change view/change members and retry," rather than waiting indefinitely. When the silent count is $\leq f$ but neither side reaches q , return `NO_QUORUM(reason=waiting)`, indicating "a few votes short, can wait one more round." This distinction separates "truly cannot gather (change view)" from "just not finished voting (wait more)," avoiding mistaking recoverable waiting for a system freeze.

4.4 The safety gate at the constructor layer

`StopConsensus.__init__` directly raises `ValueError("unsafe committee: n=...")` when `len(voters) < 3*f+1`. This advances "illegal configuration" from runtime behavior to construction time — you simply cannot construct a committee with $n < 3f+1$.

5 Experimental Setup

- **Configuration:** main test $n=7, f=2$ ($q=5$); also test illegal $n=6, f=2$ (should be rejected).
- **Fault-strategy family:** constant reverse voting, per-round swinging (equivocation), silence, unknown voter.
- **Evidence grade:** 10 deterministic property tests `verified`; end-to-end integration `cpu-proto`.
- **Hardware fingerprint:** Linux CPU, pure Python control flow, no network, no LLM calls.
- **Reproduction:** `pytest tests7/test_v747_consensus.py -q`.

Why use deterministic tests rather than random fuzzing. The safety invariant (at most one decision value) is a property that can be overturned by a counterexample: as long as one voting combination makes both sides simultaneously reach quorum, safety is broken. For such a property, **deterministic enumeration of hand-crafted counterexamples** is more direct than random fuzzing — we can precisely construct the "most dangerous combination" (e.g., 5 honest STOP vs 2 Byzantine CONTINUE) and verify it does not break. Liveness (no hang) needs count distributions, left to a random grid. At the current stage this paper mainly uses deterministic property tests, nailing down "safety-counterexample existence," and lists "liveness-round distribution" as to be supplemented.

Voter naming and the Sybil boundary. Tests use `qa-0...qa-6` as voters; test #9 uses a "sybil" vote not on the list, verifying the constructor rejects unknown persons at `cast`. This corresponds to classical BFT identity registration: only registered committee members have voting rights, preventing Sybils from injecting extra votes through forged identities.

6 Results

6.1 Safe configuration space

Figure P2-1: Safe configuration space

Figure P2-1 (restated) The n - f safe configuration space and the critical line $n=3f+1$. The horizontal axis is the Byzantine member count f , and the vertical axis is committee size n ; the blue region is the feasible region ($n \geq 3f+1$, Eq. 1), and the red region is the constructor-rejection region ($n < 3f+1$). Above the line honest members $n-f \geq 2f+1=q$, sufficient to hold the unique decision value under Byzantine reverse voting. Three candidate operating points are marked: $n=7, f=2$ (main use of this paper, $q=5$), $n=10, f=3$, $n=13, f=4$; the tolerated Byzantine counts are 2, 3, 4 in order, and each additional tolerated Byzantine costs the committee at least 3 more people. Source: the constructor-rejection rule of `udos7/topology/consensus.py`, `cpu-proto`, geometric schematic (not data scatter).

Figure P2-1 draws the safe region of n versus f : blue is the $n \geq 3f+1$ feasible region, red is $n < 3f+1$ (constructor rejection). This paper's main operating point $n=7, f=2$ falls within the feasible region.

Key reading points: the horizontal axis is Byzantine member count f , the vertical axis is committee size n . The critical line $n=3f+1$ divides the plane in two: above the line honest members $n-f \geq 2f+1$,

sufficient to hold the unique decision under Byzantine reverse voting; below the line honest votes are insufficient and safety cannot be guaranteed, so the constructor directly rejects. Three candidate operating points are marked: $n=7, f=2$ (main use), $n=10, f=3$, $n=13, f=4$. They are all within the feasible region, tolerating 2, 3, 4 Byzantine members in order. This figure also intuitively shows the "cost-fault tolerance" trade-off: to tolerate one more Byzantine member, the committee needs at least 3 more people (from 7 to 10 to go from tolerating 2 to tolerating 3).

6.2 Ten property tests (H1/H2/H3)

Table P2-1 Ten property tests and assertions (source `tests7/test_v747_consensus.py`)

#	Test	Configuration/fault injection	Assertion
1	Illegal-committee rejection	$n=6, f=2$ ($n < 7$)	Constructor raises <code>ValueError</code> (H3)
2	Honest-supermajority stop	5/7 vote STOP	<code>decision=STOP, reason=quorum</code>
3	Byzantine inability to force stop	5 honest CONTINUE + 2 Byzantine STOP	<code>decision=CONTINUE, STOP votes=2</code>
4	Equivocation detected and voided	4 STOP + b1 double vote + b2 STOP	b1 detected, STOP=5 still quorum $\rightarrow$ STOP
5	Equivocation breaks false quorum	4 STOP + b double vote	after b voided $4 < 5 \rightarrow$ NO_QUORUM
6	Conflicting quorums unreachable	5 STOP vs 2 CONTINUE	<code>not safety_violation, STOP</code>
7	Excess silence triggers view change	3 CONTINUE + 4 silent	<code>NO_QUORUM, reason=view_change</code>
8	Wait when quorum not reached	4 CONTINUE + 1 STOP, 2 silent $\leq f$	<code>NO_QUORUM, reason=waiting</code>
9	Unknown-voter rejection	Non-committee "sybil" votes	Raises <code>ValueError</code>
10	Signature determinism	Recompute same input	Same input same signature, different decision different signature

These ten cover three fault classes: false reporting (#3), swing/double voting (#4, #5), silence (#7, #8), plus configuration gate (#1), Sybil (#9), and signature determinism (#10). The key counterexample design is #3: when 5 honest members all vote CONTINUE and 2 Byzantine members maliciously vote STOP, the committee correctly returns CONTINUE — Byzantines cannot wrongly stop the order by reverse voting. #5 verifies the destructive power of equivocation: a double voter could have gathered a false quorum, but after whole-round voiding the quorum is broken and turns to NO_QUORUM.

Table P2-3 Fault-strategy family and expected decisions ($n=7, f=2, q=5$)

Fault strategy	Honest-vote distribution	Byzantine behavior	Expected decision	Corresponding test
No fault	5 STOP	None	STOP (quorum)	#2
Constant reverse vote	5 CONTINUE	2 vote STOP	CONTINUE (Byzantines blocked by majority)	#3
Swing to false quorum	4 STOP	1 double votes STOP/ CONTINUE	STOP (still 5 votes after voiding)	#4
Swing breaks quorum	4 STOP	1 double votes	NO_QUORUM ($4 < 5$ after voiding)	#5
Split votes on two sides	5 STOP / 2 CONTINUE	None	STOP (no safety_violation)	#6
Over-f silence	3 CONTINUE	4 silent	NO_QUORUM (view_change)	#7
Quorum not reached	4 CONTINUE+1 STOP	2 silent $\leq f$	NO_QUORUM (waiting)	#8
Sybil	Legal votes	Non-committee votes	ValueError (rejected)	#9

This table maps "fault strategy $\rightarrow$ expected decision" one to one and is the core of pre-registration: the expected decision in each cell is frozen before running the experiment, and the experiment only verifies conformance. If any cell deviates from expectation (e.g., returning STOP under constant reverse voting), H1 falsification is triggered.

Case-by-case walkthrough: why #3 is key. Taking #3 "Byzantine inability to force stop": 5 honest committee members all vote CONTINUE (believe not done, want rework), and 2 Byzantine committee members maliciously vote STOP (want to wrongly accept the not-done order). The committee counts: honest CONTINUE=5, Byzantine STOP=2. Quorum $\$q=5\$$. CONTINUE reaches 5, so return CONTINUE. The Byzantines' 2 STOP votes are only 2, far below $\$q=5\$$, unable to form a STOP quorum. The meaning of this test is that it directly presents the most dangerous scenario of "erroneous stop" (safety violation), verifying the committee is not misled by a few reverse votes. If BFT-lite wrote the quorum wrong (e.g., as $\$2f=4\$$), then 2 Byzantine votes could gather a false quorum — which is exactly why the quorum must be $\$2f+1\$$ rather than $\$2f\$$, and why we nail this test as a regression assertion.

6.3 End-to-end integration (CPU matrix)

Figure P2-2: Clean vs stressed fleet

Figure P2-2 (restated) End-to-end matrix integration: clean fleet vs stressed fleet ($\$n=7,f=2\$$ QA committee). Grouped bars: each group has two bars, blue = clean fleet, orange = stressed fleet (2 Byzantine QA reverse voters + 3 faulty workers); the vertical axis is the corresponding metric value, groups from left to right being accepted (both 24/24), QA rounds (24 vs 25), retries (0 vs 5), board_messages (48 vs 54). The stressed fleet has 1 more QA round, 5 reworks, and 6 board messages, but the acceptance rate does not fall. Data values come item by item from the `clean.*` and `stressed.*` of `reports7/matrix_scale_demo.json` (cpu-proto, single seed=1); the figure is a clean-vs-stressed comparison, with Byzantine reverse votes blocked by the honest majority.

Table P2-2 End-to-end matrix integration (source `reports7/matrix_scale_demo.json`)

Metric	Clean fleet	Stressed fleet (f=2 Byzantine QA + faulty workers)
Orders n_orders	24	24
Accepted	24	24
QA rounds qa_rounds	24	25
Retries	0	5
Isolated agents	[]	[spec-kinematics-0, spec-spatial-1, spec-dataflywheel-2]
Board messages board_messages	48	54

In the matrix, the 7-member QA committee (`qa_voters=7`, `qa_f=2`) votes on each order: the 2 Byzantine members are set to "the reverse direction of the honest result" (`vote = not honest_accept`). Even so, all 24 orders are accepted correctly — the Byzantine specialists' wrong results are blocked by the majority. The stressed fleet triggers 5 more reworks (`retries=5`) and isolates 3 faulty agents with accumulated quality failures (`isolated_agents`); the governance audit `governance.ok=true` and Trace is complete. This shows BFT-lite in the real matrix loop not only "passes tests" but "blocks reverse votes and triggers rework/isolation."

How rework and isolation cooperate. It is worth taking apart the role BFT-lite plays in the matrix loop. After each order is completed, the specialist produces a result, and the QA committee votes on "pass (STOP)/rework (CONTINUE)." When the majority judges CONTINUE, the order is re-posted to the blackboard with `key:rework{round_id}` and one quality failure is recorded; when accumulated failures trigger the circuit breaker, the corresponding specialist is isolated (`isolated_agents`). The clean fleet's 24 orders happen to have 24 QA rounds and 0 reworks; the stressed fleet has 25 QA rounds, 5 reworks, and 3 isolations. This shows BFT-lite is not an isolated "voting toy" but is embedded in the governance loop of "acceptance → rework → isolation": it decides when to stop, while the governance trio (circuit breaker/rollback/reassignment) decides how to handle after stopping. The Byzantine QA's reverse votes are blocked by the majority, letting no defective order be erroneously stopped.

Interpretation of the difference from the clean fleet. The clean fleet has 24 orders, 24 QA rounds, 0 reworks; the stressed fleet has 25 QA rounds, 5 reworks. The extra 1 QA round and 5 reworks are precisely the cost of Byzantine reverse voting and faulty workers; the committee does not "wipe away" these costs but honestly reflects them in the rework count and isolation list. This is important: BFT-lite is not "free fault tolerance"; its fault-tolerance cost is more QA rounds and reworks; what it guarantees is "not stopping wrongly," not "zero cost." Board messages growing from 48 to 54 also reflect the extra coordination overhead from rework.

7 Discussion and Threat Validity

Signatures are hash placeholders, not cryptographically secure. `sign` is only SHA-256 truncation and provides no forgery resistance. In a real network an attacker can forge the hash of any (`voter`, `decision`, `round_id`). This paper therefore does **not** claim resistance to real forgery, only that "under controlled CPU simulation, voting-record determinism and equivocation detection are correct."

Network partitions and adaptive adversaries are not modeled. This paper assumes all votes are delivered within the same round, with no partition and no message reordering; Byzantine members are static (fixed at 2), not adaptive corruption. Network partitions and adaptive adversaries in real clusters require cluster fault injection (Gate D).

Semantic common-cause failure is the biggest threat. Classical BFT assumes errors are independent and random; but agent wrong votes have semantic correlation. If $>f$ QA members are systematically misled by the **same erroneous context** (such as the same defective Transfer Bundle), then even at $\frac{n}{3}+1$ the majority vote will be consistently wrong — at this point the BFT safety assumption fails. This is exactly the intersection with P10 (Agent-as-Tool / Transfer Bundle information loss): common-cause failures cannot be resolved by voting majority, only by cutting the common cause. This paper lists it as future work and does not pretend coverage.

The asymmetric trade-off of safety and liveness. This paper's design implies a trade-off: be conservative on safety — whole-round voiding of equivocation may make a round undecidable (liveness cost), but this avoids a swinger gathering a false quorum (safety benefit). This "safety over liveness" trade-off is consistent with classical BFT, but in the agent scenario it is concretized as: rather rework one more round (view change) than let a defective order be wrongly accepted. Whether this trade-off is always optimal depends on the cost comparison between "erroneous stop" and "excessive waiting"; in acceptance scenarios the cost of erroneous stop is usually higher, so the trade-off is reasonable, but we do not claim it holds for all scenarios.

Safety assertions are deterministic and need no p-value. Unlike the statistical experiments of P4/P9, H1 is a deterministic assertion: expected counterexample count is 0. We report "exhaustion/property-test coverage," not confidence intervals. The pre-registration rule is "finding any safety counterexample reverses the paper's conclusion, without tuning parameters to remove it."

The liveness-round distribution has not yet had a random grid. The current 10 tests are hand-designed representative scenarios, not an exhaustive statistic over an " n, f grid $\times$ random fault configurations." The one-pager plans $n=7/10/13$ with the corresponding f grids each having ≥ 1000 random fault configurations, reporting the median and p95 of termination rounds. This step has not run and is marked [RESULT NEEDED: termination-round distribution over $n \times f$ grid with ≥ 1000 random configurations]. Before completing it, we only claim "the liveness invariant holds under representative scenarios," not "liveness is bounded under all random configurations."

The comparison with single-point QA is missing. This paper tested BFT-lite, but the two comparisons planned in the one-pager — no consensus (single-point QA) and naive majority vote (no equivocation detection) — have not yet had full-factorial comparison experiments in the matrix. This means "how much better BFT-lite is than a single point" currently has only a mechanism argument and no comparison data. Completing this comparison is a key experiment before submission.

8 Resource Gates and Applicability Boundaries

Currently reachable (CPU, completed in this paper): BFT-lite committee, whole-round voiding of equivocation, view change, 10 property tests, end-to-end 7-member QA integration.

Unlock items: real-network robustness (partitions, reordering, adaptive adversaries) requires cluster fault injection (Gate D); forgery resistance requires real cryptographic-signature infrastructure.

Applicability boundaries: this paper applies to "narrow-domain, semantic-judgment stop decisions" (acceptance/rework) and does not claim to apply to open consensus or fund settlement.

Future work: common-cause failure and Transfer Bundle. This paper's biggest uncovered threat is semantic common-cause failure. If $>f$ QA committee members see the same defective context (e.g., the same information-losing Transfer Bundle), they will be consistently wrong — at this point the voting majority instead amplifies the error. The solution direction is not enlarging the committee but cutting

the common cause: letting different members see independent, de-biased contexts (which is exactly P10's intersection). Combining "BFT majority" with "context de-biasing" is the next step.

Future work: from property tests to formalization. The current 10 items are hand-crafted counterexamples. The next step can use model checking/symbolic execution to exhaust all voting combinations for $n=7$, giving an exhaustive proof of "zero safety counterexamples," rather than just "10 representative scenarios do not break." This upgrades "representative" to "exhaustive."

9 Conclusion

For multi-agent stop decisions, BFT-lite uses $n \geq f+1$, $2f+1$ quorum, whole-round voiding of equivocation, and over- f silence triggering a view change to simultaneously hold "not stopping wrongly" and "not waiting indefinitely" under controlled CPU simulation. The 10 property tests fix these properties as falsifiable regression assertions, and end-to-end integration holds 24/24 acceptance under 2 Byzantine reverse voters and triggers rework/isolation. This paper's stance is that for agent stop decisions, safety/liveness should be presented as executable property tests rather than only prose proofs; while honestly stating the three boundaries of hash-placeholder signatures, unmodeled partitions, and semantic common-cause failures.

One-sentence conclusion: when the "when to stop" of a million-agent matrix cannot be given to a single point, a lightweight committee satisfying $n \geq f+1$, $2f+1$ quorum, whole-round voiding of same-round swingers, and over- f silence triggering a view change can simultaneously hold "not wrongly accepting" and "not waiting indefinitely" under controlled CPU simulation; these properties are supported not by slogans but by 10 falsifiable property tests and end-to-end 24/24 integration. The next step needs the random-fault grid for liveness rounds, full-factorial comparison with single-point/naive majority, and honest recording of semantic common-cause failure as an open threat. All figures in this paper can be sourced line by line from `consensus.py`, `test_v747_consensus.py`, and `matrix_scale_demo.json`, relying on no external dataset or vendor scope, which is why it can serve as a "reproducible safety verification" sample, suitable directly as a teaching and regression baseline.

One-sentence advice for "agent reliability" engineering. At the end of a multi-agent pipeline, do not give "stop/rework" to a single-point QA; use a committee satisfying $n \geq f+1$, void swingers for the whole round, trigger a view change for over- f silence, and write these properties as CI-able property tests. Fault tolerance is not free — its cost is more QA rounds and reworks — but what it holds is the bottom line of "not letting a defective deliverable be wrongly accepted." All conclusions of this paper are bounded within the CPU deterministic prototype and hash-placeholder signatures, and real-network robustness and forgery resistance are the next stage requiring extra infrastructure. We do not repackage classical BFT as a new algorithm, but honestly land it on the new end of agent stop decisions and hold the two bottom lines of safety/liveness with falsifiable property tests and end-to-end integration. For engineering teams, three things are directly reusable: a minimal committee configuration with $n \geq f+1$, the decision rules of whole-round voiding of same-round swingers and over- f silence triggering view change, and a CI-able property-test suite. For researchers, the open problem this paper wants to leave is semantic common-cause failure — it reminds us that majority votes do not equal correctness, and when all committee members share the same defective context, fault tolerance is broken through by the common cause — which is the problem this paper most wants to leave to subsequent work.

Relation to other papers in the paper matrix. This paper (P2) covers fault tolerance of the end stop decision, and together with P1 (communication topology), P3 (governance trio), and P10 (handoff

information loss) forms one piece of the agent matrix's reliable operation: P1 solves "how to communicate," P2 solves "who decides to stop," P3 solves "how to handle problems after stopping," and P10 solves "whether handoff context introduces common-cause failure." This paper's biggest open threat (semantic common-cause failure) is precisely P10's interface — which is why we list it as future work rather than pretending coverage.

References

The first three items below are  verified entries in this shared "Master Reference Library" (verification date 2026-09-19, fields copied from the master library); the rest are search directions to be supplemented, kept as placeholders, not fabricated.

Verified (2026-09-19)

1. Castro, M., Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999 (USENIX). (Related: tolerates $\frac{n}{3}$ Byzantine nodes under asynchrony, pre-prepare/prepare/commit three phases, BFS only about 3% slower than unreplicated NFS; the direct source of this paper's $2f+1$, $2f+1$ quorum.)
2. Yin, M., Malkhi, D., Reiter, M. K., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain. PODC 2019. arXiv:1803.05069. (Related: linear communication complexity, responsiveness, chained view changes; reference for this paper's view-change skeleton.)
3. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (Related: a conversable multi-agent framework, the comparison for this paper's agent stop-decision scenario; its "dialogue without termination conditions" limitation is precisely this paper's entry point.)

To be verified (keeping [CITATION NEEDED] , not fabricated)

- LLM voting/multi-agent decision: LLM multi-agent voting decision byzantine.
- Property tests/fault injection: property-based testing hypothesis fault injection invariant.

Appendix

Appendix A Reproduction commands and evidence ledger

```
pytest tests7/test_v747_consensus.py -q      # 10 items
python scripts7/matrix_demo.py              # end-to-end matrix demo
```

Evidence ledger: source `udos7/topology/consensus.py`; tests `tests7/test_v747_consensus.py` (10 items); integration data `reports7/matrix_scale_demo.json` (cpu-proto); matrix configuration `qa_voters=7, qa_f=2` (`udos7/topology/matrix.py`). The full regression `pytest tests7/ --collect-only -q` is 236 items (2 skipped), and commit `1a71270` records all green.

Key code-entity index. `StopConsensus` (committee class), `cast` (vote), `equivocators` (swing detection), `decide` (decision), `ConsensusResult` (six fields `decision/quorum/equivocators/silent/safety_violation/reason`), `sign` (hash placeholder). The three decision states `STOP/CONTINUE/NO_QUORUM` and reasons `quorum/view_change/waiting/conflicting_quorums`. These entity names and fields are actually consumed in the end-to-

end calls of `matrix.py: cons.cast(v, STOP if vote else CONTINUE, round_id)`, accepting and settling when `res.decision == STOP`, otherwise reworking.

Appendix B Number traceability spot checks

1. Illegal-rejection threshold: $n < 3f + 1 = 7$ rejected $\leftarrow$ `consensus.py __init__`.
2. Quorum $q = 2f + 1 = 5$ $\leftarrow$ `self.quorum_size`.
3. Stressed fleet accepted = 24 $\leftarrow$ `matrix_scale_demo.json stressed.accepted`.
4. Retries = 5 $\leftarrow$ `stressed.retries`.
5. Isolation of 3 agents $\leftarrow$ `stressed.isolated_agents`.

Appendix C Author's intended-use statement

- **Target outlets:** OPODIS / ICDCS workshops, AAMAS, DSN workshop. Quartile/IF/deadline [to be verified], verified online before submission with the date marked.
- **Pre-registration plan:** safety assertions deterministic (0 counterexamples expected); liveness rounds as count distributions, n, f grids each ≥ 1000 random fault configurations reporting median/p95; "finding any safety counterexample reverses the conclusion."
- **Data and code availability:** Apache-2.0; tag v7.5.0 (commit 1a71270); report `reports7/matrix_scale_demo.json`.
- **AI-use statement:** AI assisted in generating the draft and figure scripts, all figures sourced and checked by the author from source/JSON; hash-placeholder signatures and unmodeled partitions/common-cause failures are explicitly stated; unverified p-values/DOIs/quartiles were not filled in.

Appendix D Internal review record (five-dimension reviewer self-assessment, look only, do not change)

1. **Novelty:** 3/5. BFT itself is mature; the novelty lies in the application face of "agent stop decision + executable property tests."
2. **Evidence strength:** 4/5. Safety assertions are deterministic tests (10 verified); the liveness distribution has not yet had a ≥ 1000 random-fault grid (marked [RESULT NEEDED]).
3. **Reproducibility:** 5/5. Pure CPU, deterministic, reproduction commands, complete test index.
4. **External validity:** 2/5. Hash-placeholder signatures, no network partitions, semantic common-cause failures not modeled (stated).
5. **Writing and honesty:** 5/5. Safety/liveness counterexample design, clear boundaries and future work.

Overall judgment: a reliable-systems paper with high engineering-teaching value; before submission it needs the random-fault grid statistics for liveness rounds and a clear statement of differences from classical PBFT to avoid being judged incremental.